

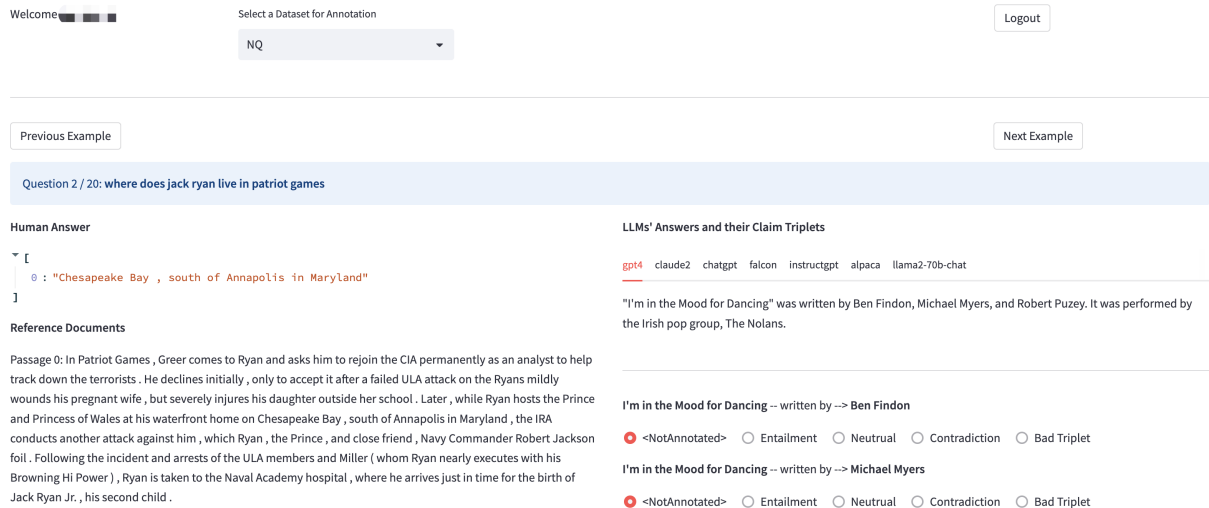


Figure 10: The screenshot of the annotation tool for human evaluation.

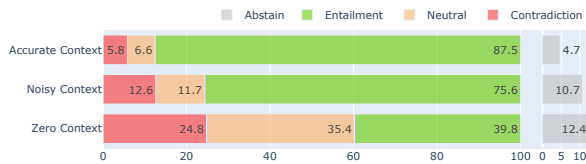


Figure 11: Results of different task settings by averaging the results of the seven LLMs.

Copy from Context is Safer Replicating content in the context enhances the factuality, as illustrated in Figure 13. In order to quantitatively assess the relationship between copying and hallucination in both Noisy and Accurate Context settings, we introduce the concept of *Copy Rate*. This metric is defined as the ratio of N-grams covered by the context, where an N-gram refers to a phrase comprising N consecutive words. Specifically, we compute the average copy rates for 1 to 4 grams of a claim-triplet to determine its overall copy rate. The findings presented in Figure 14 reveal a discernible trend: a higher copy rate corresponds to an increased likelihood of entailment.

B Details of RefChecker

B.1 Extractor

The prompts used for few-shot claim extraction are shown in Figure 15. They are used for claim extraction by GPT-4, Claude 2, Mistral, and the Mistral baseline. For Mistral-SFT, we removed the in-context examples in the prompt because we find it doesn't affect the extraction quality after supervised fine-tuning but saves context length.

We collected 10,000 questions without claim ex-

	Response	Sentence	Sub-sentence	Triplet
RoBERTa-NLI	44.92	51.97	50.18	55.19
AlignScore	46.05	53.19	50.71	57.60
GPT4	55.86	56.66	47.50	58.78
Claude2	46.49	46.67	34.57	41.00
RepC-LE-knn	45.36	50.79	46.29	55.33
RepC-LE-svm	48.91	54.26	52.29	59.81
RepC-LE-nn	44.03	53.26	50.83	57.54

Table 7: Detailed performance of 7 checkers under different claim granularities on 2.1k manual annotated responses. The checkers' predictions under different granularities are all merged into response-level and then evaluated.

traction results and annotation, following the same process as described in Appendix A. The collected questions cover the three context settings evenly. We collected responses to those questions by Mistral and queried Mistral 8x7B to get corresponding claims. After that, we performed supervised fine-tuning on a Mistral 7B model to distill the output of the larger Mistral model. We trained the model for 1 epoch with a initial learning rate 1e-5.

B.2 Checker

The prompts used for the GPT-4 and Claude 2 based checkers are shown in Figure 16.

As a supplement of Figure 5, Table 7 shows the detailed checker performance under different claim granularities. As a supplement of Table 5, Table 8 shows the full results of checker evaluation.

In the analysis of Table 5, we claim that Claude 2 checker tends to flag a neutral claim as contradiction. We can observe such tendency in Table 9, especially for MS MARCO and Dolly datasets.

We also study the performance tendency of

Models	Average of three settings		Zero-context (NQ)		Noisy-context (MS MARCO)		Accurate-context (databricks-dolly-15k)	
	Accuracy	Macro-F1	Accuracy	Macro-F1	Accuracy	Macro-F1	Accuracy	Macro-F1
Baseline Checkers								
RoBERTa-NLI	76.56	55.88	74.06	<u>69.90</u>	78.36	46.67	77.27	51.06
AlignScore	78.85	<u>59.45</u>	73.40	70.28	78.86	<u>50.42</u>	84.30	<u>57.66</u>
GPT-4	74.77	59.80	67.46	66.10	76.67	55.49	80.17	57.80
Claude 2	51.98	36.55	43.42	42.90	40.35	25.89	72.18	40.87
Mistral-based Checkers								
zero-shot	69.43	46.64	70.83	61.10	71.75	43.01	65.72	35.81
1-shot	76.68	50.66	65.44	63.25	81.23	42.18	83.38	46.56
3-shot	74.24	45.89	56.67	56.20	81.55	37.41	84.50	44.07
LoRA-sft-n2000	72.06	52.62	74.09	68.22	75.20	48.65	66.90	40.99
LoRA-sft-n4000	77.84	57.98	77.43	<u>73.64</u>	79.21	<u>50.29</u>	76.89	50.00
RepC-LS-knn-n100	74.36	51.98	72.67	68.58	77.54	45.19	72.86	42.17
RepC-LE-knn-n100-e100	69.72	51.64	70.26	66.05	71.14	46.33	67.75	42.55
RepC-LS-svm-n1000	79.15	59.36	78.34	74.04	79.82	47.62	79.29	<u>56.43</u>
RepC-LE-svm-n1000-e1000	79.03	<u>60.05</u>	77.98	73.53	79.56	51.29	79.54	55.34
RepC-LS-nn-n2000	80.17	57.31	75.50	71.95	81.78	46.90	83.22	53.07
RepC-LE-nn-n2000-e2000	81.27	60.80	75.23	71.98	82.08	47.56	86.50	62.86

Table 8: Checker evaluation results on 11k human annotated claim triplets. In Mistral-based checkers, the model names start with the variant types, eg. LoRA-sft indicates the LoRA fine-tuned variant and RepC-LE-nn indicates the representation based classification variant using layer ensemble with 2-layer MLP as the classifier. Here “nxxx” and “exxx” indicates the number of training samples and ensemble learning samples.

	NQ	MS MARCO	Dolly	Total
# triplets	3319	3420	3994	10733
# neutral labels	1818	436	368	2622
# pred as E	313	88	165	566
# pred as N	201	9	16	226
# pred as C	1304	339	187	1830

Table 9: Detailed statistics of the Claude 2 checker’s neutral F1 score. We show its prediction results of all neutral labels.

RepC-LS and RepC-LE in Figure 17. The findings indicate that in RepC-LS, the best performed layer is typically around the middle rather than the last layer. Despite RepC-LS trailing behind RepC-LE, it maintains its advantages in model size and data efficiency.

Besides, in Figure 18, we evaluate the RepC checker performance with respect to different number of training data. We can see that RepC-LS-svm outperforms RepC-LS-nn with fewer training data.

B.3 Source Attribution

In many cases, users of hallucination detection systems care not only the verdicts of the checker, but also where the hallucination happens in the original response, as well as which evidence in the reference supports such verdicts. We provided a rudimentary support of such demand. Specifically, we apply a sentence embedding model (SimCSE (Gao et al., 2021)) to encode spans in responses and references,

compare them to the encoding of elements in claim-triplets, then use a threshold to filter matched spans as source attribution results. This naive approach suffers from issues on computational efficiency, unclear boundaries, and matching by shallow semantics. The topic on source attribution has a significant impact on applications of hallucination detection and we leave the exploration on non-trivial solutions to the future.

C Comparisons with Other Hallucination Detection Frameworks

C.1 Comparison on the REFCHECKER Benchmark

We compare REFCHECKER with recently proposed hallucination detection frameworks, SelfCheckGPT, FActScore and FacTool, on our benchmark. The four frameworks use different representations of claims and hallucination labels as described in Table 1, we aggregate the claim-level results into two types of response-level results:

- Response-level binary classification. We aggregate the claim-level labels into response-level binary labels as Factual and Non-Factual. Thus, we use Accuracy, Factual F1 (Fact. F1 for short) and Non-Factual F1 (Non-Fact. F1) as the evaluation metrics. We use a strict configuration that a response is non-factual if at least one claim contains hallu-

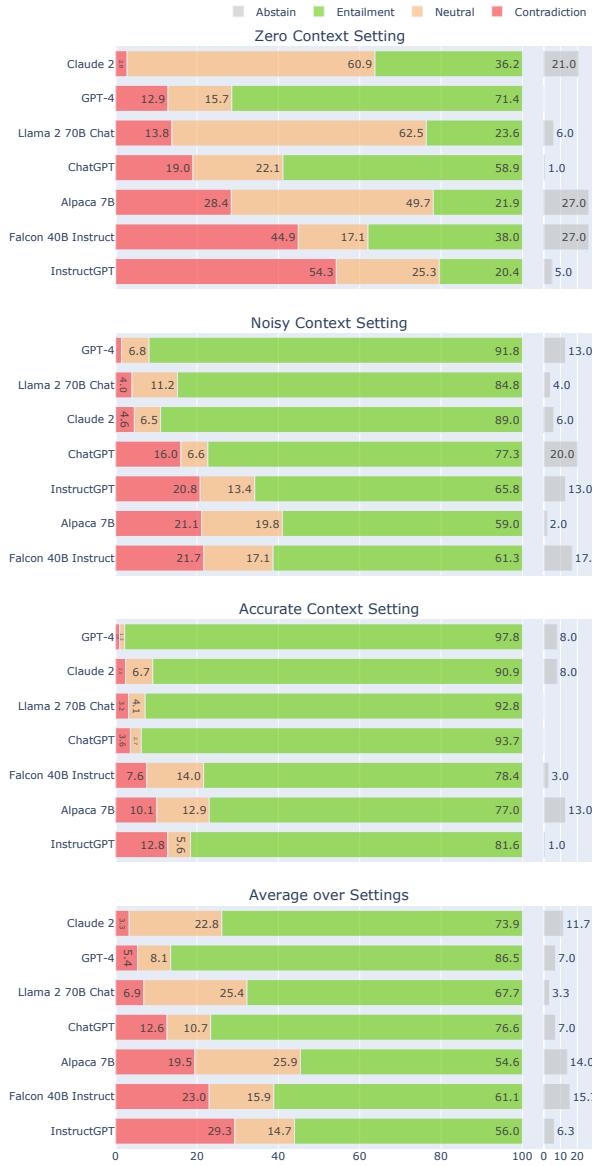


Figure 12: Detailed results of the human evaluation with the rates of Contradiction, Neutral, Entailment and also Abstain across the three settings, and ranked by the contradiction rates. The last ranking is the results averaged over the three settings.

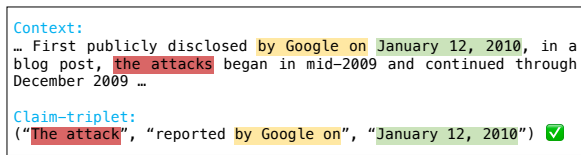


Figure 13: An example of factual claim-triplet whose content are mostly copied from the context.



Figure 14: Copy rate of the claim-triplet v.s. label distributions, by aggregating the results of the Noisy Context and Accurate Context settings.

cination. For SelfCheckGPT, we consider `minor_inaccurate` and `major_inaccurate` labels as hallucination. For RefChecker, we consider both Contradiction and Neutral as hallucination.

- Correlations of response-level hallucination rate. Following SelfCheckGPT, we also compare the hallucination rate of a response with human evaluation by Pearson and Spearman correlations. For SelfCheckGPT, we compute the hallucination rate of a response by averaging the scores of the sentences following the definition in their paper. For FactScore and FacTool, the hallucination rate is the ratio of non-factual claims in a response. And for RefChecker, we take the ratio of Contradiction and Neutral claims as the hallucination rate.

The results are shown in Table 10 for Zero Context setting, Table 11 for Noisy Context setting and Table 12 for Accurate Context setting. Following their configurations in their papers, we apply InstructGPT(text-davinci-003) and GPT-4 as the extractors for FactScore and FacTool, respectively, apply ChatGPT(gpt-3.5-turbo) as the checker for SelfCheckGPT and FactScore and GPT-4 for FacTool. The combinations of extractor and checker of RefChecker are displayed as “{Extractor} + {Checker}”.

We conclude these results with the following observations:

- RefChecker is effective. Most combinations